Challenge 1-a: RFP Tool - Proposal Management - UI Prototype

This challenge implements the frontend UI for the "Proposal Management" feature as a new self-contained app module in the `platform-ui` project. The feature provides a split-view interface with a sidebar for proposal navigation and a main content area with "Build" and "Review" tabs, enabling users to manage RFP proposals through a guided workflow.

This is a **frontend-only** challenge. All backend API calls must be stubbed with mock data and mock service functions, allowing the full UI workflow to be demonstrated and tested without a running backend.

Design Reference:

- Wireframes: See provided `RFP_Wireframes.pptx` with annotations (16 slides covering all elements numbered 1–30)

Technology Stack

- **Frontend**: React 18, TypeScript 4.9
- **State Management**: Redux with redux-thunk; SWR for data fetching
- **Routing**: React Router v6
- **Styling**: CSS Modules with SCSS
- **Forms**: React Hook Form with Yup validation
- **Testing**: Jest with React Testing Library
- **Build Tools**: Craco (Create React App Configuration Override)

Important: Must match the existing technology stack and patterns used in the `platform-ui` project. Reuse the in-house UI component library at `src/libs/ui` and shared utilities at `src/libs/shared`. Follow the module structure established by existing apps (e.g., `src/apps/engagements/`, `src/apps/talent-search/`).

Required links

- platform-ui develop branch

Assets

- **Wireframes**: `RFP_Wireframes.pptx` — 16 slides with element annotations (Elements 1–30). Key wireframe views:
  - Slide 2: Main Tab Layout — shows the sidebar (Elements 3, 4, 5, 6, 7, 30) and content window (Element 8) with top-level tabs (Elements 1, 2)
  - Slide 3: Content Layout – Build View — shows the Build tab (Element 9) with Documents Panel (11), Request Panel (12), and Q&A Panel (13)
  - Slide 4: Build View – Documents Panel — shows Document Info (Element 19) with file names listed and an "Add Document" button (Element 20)
  - Slide 5: Build View – Request Panel — shows the RFP summary header (Element 21), text area (Element 22), and "Assess" button (Element 23)
  - Slide 6: Build View – Q&A Panel — shows Question header (Element 24), individual questions (Element 25), answer text areas (Element 26), and "Answer" button (Element 27)
  - Slide 7: Content Tab Layout – Review View — shows the Review tab (Element 10) with Context Refresh Timer (Element 29), informational message (Element 18), Proposal PDF area (Element 17), and "Request Quote" button (Element 28)
  - Slides 8–16: Element Functionality descriptions — detailed behavior specifications for all 30 elements
- **Existing Codebase**: `platform-ui/src/`
  - `src/apps/` - Feature modules (reference `engagements/` and `talent-search/` for patterns)
  - `src/libs/ui/` - Shared UI component library
  - `src/libs/core/` - Core utilities (auth, XHR, routing)
  - `src/libs/shared/` - Shared hooks, contexts, services

Individual Requirements

1. Module Structure

Create a new `rfp-tool` app module as a self-contained feature under `src/apps/rfp-tool/`. This module will house both the Proposal Management functionality (this challenge) and the Admin Challenge Curation functionality (future Challenge 2).

1.1 File Structure

Create the following structure:

```
src/apps/rfp-tool/
├── pages/
│   └── RfpToolPage.tsx            # Main page component (top-level tabs: Proposal
Management, Admin)
├── lib/
│   ├── components/
│   ├── models/
│   ├── services/
│   ├── hooks/
│   └── mock/
│       └── proposals.mock.ts      # Mock data and mock implementations
├── rfp-tool.routes.tsx            # Route definitions
└── index.ts
```

1.2 Routing Integration

- Define a `PlatformRoute` for the rfp-tool module following the pattern in `src/apps/talent-search/src/talent-search.routes.tsx`
- Register the route in `src/apps/platform/src/platform.routes.tsx` by importing and spreading into the `platformRoutes` array
- Use lazy loading via the existing `lazyLoad` utility consistent with other app modules
- The route registration in `platform.routes.tsx` is the **only** file outside of `src/apps/rfp-tool/` that should be modified

2. Layout and Navigation

Implement the split-view interface as shown in wireframe Slides 2–3.

2.1 Sidebar & Proposal List (Elements 3–7, 30)

Implement the left-hand sidebar to manage the list of proposals.

- **Header**: Display "Proposals" header (Element 3), positioned above the list and to the left of the "New" button (Element 30)
- **List View** (Elements 4–6):
  - Show a scrollable list of existing proposals with a scrollbar (Element 7)
  - The active/selected proposal must be visually highlighted (as shown in wireframe with Element 5 highlighted in green)
  - Clicking an inactive proposal (e.g., Element 4 "Benchmarking Util") loads it into the Content Window (Element 8)
- **New Proposal** (Element 30):
  - Implement the "New" button next to the "Proposals" header

- Action: On click, show a popup modal asking for a "Proposal Name" — use the existing **ConfirmModal** component from `src/libs/ui`
- Logic: Call the mock create service immediately upon confirmation
- Constraint: Proposal names do not need to be unique; each proposal receives a unique UUID (per Element 30 description)

2.2 Main Content Area Tabs (Elements 1, 2, 8)

The main Content Window (Element 8) contains the workflow.

- **Top-Level Tab Navigation** (Elements 1, 2):
  - "Proposal Management" (Element 1): The default active tab. This is the only viewable tab for V1
  - "Admin Tab" (Element 2): This tab header/button must be visible but **disabled** for this challenge. Per the wireframe: "In a future version this will allow users designated as admins to perform administrative functions." The Admin Tab will be enabled in Challenge 2 (Admin Challenge Curation), which will add its own sidebar and content views within this same `rfp-tool` module
- **Sub-Tab Navigation** (Elements 9, 10):
  - "Build" tab (Element 9): Shows the build workflow (Documents, Request, Q&A panels)
  - "Review" tab (Element 10): Shows the proposal PDF and quote request
  - Content Window updates based on which sub-tab is active
- Use the existing **TabsNavbar** / **TabsNavbarItem** components from `src/libs/ui` for tab navigation

3. Build Tab Functionality (Element 9)

The Build tab contains three vertically stacked panels within a scrollable area (Element 14 — Build Panel Scrollbar).

3.1 Documents Panel (Elements 11, 19, 20)

- **Functionality**: Allows uploading context documents for the RFP (Element 11)
- **Document Info** (Element 19): Shows the count and names of uploaded documents in a list format (as shown in wireframe Slide 4: "Overview.docx, Budget.xls, SystemReqs.pdf, FeatureList.docx, FeatureListAddl.docx, Notes.rtf")
- **Add Document Button** (Element 20): Allows the user to upload a document. Use the existing **InputFilePicker** component from `src/libs/ui` for file selection
- **File Constraints**:
  - Allowed Types: `.pdf`, `.doc`, `.txt`
  - Max Count: 10 documents per proposal (cumulative)
  - Max Size: 5MB per file
- **Validation**: Display clear error messages when constraints are violated
- **Mock behavior**: Store files in component state; simulate upload success after a brief delay

3.2 Request Panel (Elements 12, 21–23)

- **Header** (Element 21): "Please provide a summary of the RFP (under 1000 words)"
- **Input** (Element 22): Text area for user input with word count indicator. Use the existing **InputTextarea** component from `src/libs/ui`
- **Validation**: Enforce the 1000-word limit using Yup schema
- **Action**: "Assess" Button (Element 23)
  - Use the existing **Button** component from `src/libs/ui`
  - Behavior: When clicked, calls the mock assess service. Per wireframe: "After uploading documents and entering an RFP summary, the user presses this to get an initial review"
  - State Change: Triggers the "Context Refresh Timer" (Element 29, see 3.4)
  - Loading State: Show **LoadingSpinner** from `src/libs/ui` while "processing"
  - Result: Upon success, reveals the Q&A Panel (Element 13)

3.3 Q&A Panel (Elements 13, 24–27)

- **Visibility**: Hidden until the "Assess" action completes. Per wireframe: "Pressing the Assess button sends information to the back end, and when that call is completed, the Q&A panel (13) may be displayed"
- **Header** (Element 24): "Please provide answers to these clarifying questions"
- **Questions** (Element 25): Displays clarifying question text. Up to 5 questions
- **Answers** (Element 26): Text area for each answer. Use **InputTextarea** from `src/libs/ui`
- **Action**: "Answer" Button (Element 27)
  - Use **Button** from `src/libs/ui`
  - Per wireframe: "After entering all answers the user presses this button to generate a proposal, which will be visible in the Review Tab (10)"
  - Loading State: Show **LoadingSpinner** while "processing"
  - Redirect: Upon success, automatically switches the main view to the Review Tab
- **Timer interaction**: Per wireframe Element 29 description — if the Context Refresh Timer expires, the "Answer" button (Element 27) must be disabled, although the "Assess" button (Element 23) remains available to start over

3.4 Context Refresh Timer (Element 29)

- **Display**: Show a visible countdown timer in the Build tab. Per wireframe: "This is a 15 minute timer that begins when the user presses the Assess Button (23)"
- **Duration**: 15 minutes, starting when "Assess" is clicked
- **Behavior**:
  - Count down from 15:00 to 0:00
  - Per wireframe: "The user has 15 minutes to answer questions and press the Answer Button (26) or the AI context will be refreshed, and the user would have to start over"
  - When expired: disable the "Answer" button and show an expiration message. The "Assess" button remains enabled for the user to start over
  - Visual warning when < 2 minutes remain (color change or animation)
- **Persistence**: Store the timer start timestamp in localStorage so the timer survives page refreshes (simulating the server-side timestamp sync required in the Integration challenge)

4. Review Tab Functionality (Element 10)

The Review tab has its own scrollbar (Element 15 — Review Panel Scrollbar).

4.1 Proposal State (Elements 17, 18)

- **Incomplete State** (Element 18): If the proposal has not been generated (no request submitted or questions unanswered), display an informational message. Per wireframe: "If the proposal is being built (No request or unanswered questions) this message will be displayed to the user instead of the Proposal PDF (17)"
- **Complete State** (Element 17): Display the generated Proposal PDF using an embedded PDF viewer (`<iframe>` or `<object>`) or a download link

4.2 Request Quote (Element 28)

- **Button**: "Request Quote" — use **Button** from `src/libs/ui`
- **Behavior**: Per wireframe: "After reviewing the proposal, the user can make a formal request for a quote. This button is then disabled. The user will be contacted outside of this workflow"
- Calls the mock quote service. The button becomes disabled immediately after clicking and shows a "Quote Requested" confirmation state
- **Persistence**: The disabled state must persist when navigating away and back to the Review tab

5. Reusable Component Mapping

The following existing components from `src/libs/ui` and `src/libs/shared` **must** be reused:

Wireframe Element	Existing Component	Location
Top-level tabs (1, 2) and sub-tabs (9, 10)	**TabsNavbar** / **TabsNavbarItem**	`src/libs/ui/lib/components/tabs-navbar/`
New Proposal popup modal	**ConfirmModal**	`src/libs/ui/lib/components/modals/confirm/`
All buttons (20, 23, 27, 28)	**Button**	`src/libs/ui/lib/components/button/`
File upload (20)	**InputFilePicker**	`src/libs/ui/lib/components/form/form-groups/form-input/input-file-picker/`
Text areas (22, 26)	**InputTextarea**	`src/libs/ui/lib/components/form/form-groups/form-input/input-textarea/`
Text input (proposal name in modal)	**InputText**	`src/libs/ui/lib/components/form/form-groups/form-input/input-text/`
Loading indicators	**LoadingSpinner**	`src/libs/ui/lib/components/loading-spinner/`
Page layout container	**ContentLayout**	`src/libs/ui/lib/components/content-layout/`
Tooltip hints	**Tooltip**	`src/libs/ui/lib/components/tooltip/`

If an existing component does not precisely fit a wireframe requirement, extend or wrap it within `src/apps/rfp-tool/` — do **not** modify the shared component library.

6. Mock Data System

Create `src/apps/rfp-tool/lib/mock/proposals.mock.ts` to enable full UI testing without a backend. The mock data and service function signatures **must follow the API contract defined in the Backend Development challenge** so that the Integration challenge can swap mock implementations for real API calls with zero component changes.

6.1 API Contract Reference

Each mock service function corresponds to a real backend REST endpoint. The table below defines the contract that both the frontend mock and the backend must conform to. Both challenges (UI Prototype and Backend Development) run in parallel — this shared contract ensures they converge cleanly in the Integration challenge.

Service Function	Backend Endpoint	Request	Response
`getProposals()`	`GET /proposals`	—	`{ proposals: Proposal[] }` (sorted by `updatedAt` desc)
`createProposal(body)`	`POST /proposals`	`{ name: string }`	`Proposal` (full object with generated UUID, status `DRAFT`)
`getDocuments(proposalId)`	`GET /proposals/{id}/documents`	—	`ProposalDocument[]`
`uploadDocuments(proposalId, files)`	`POST /proposals/{id}/documents`	`multipart/form-data` (field: `files`)	`ProposalDocument[]` (all documents for the proposal)

Service Function	Backend Endpoint	Request	Response
`assessProposal(proposalId, body)`	`POST /proposals/{id}/assess`	`{ summary: string }`	`{ questions: string[], stub: string, timerStartedAt: string }`
`answerQuestions(proposalId, body)`	`POST /proposals/{id}/answer`	`{ answers: string[] }`	`{ pdfUrl: string }`
`requestQuote(proposalId)`	`POST /proposals/{id}/quote`	—	`{ id: string, status: "QUOTE_REQUESTED", quoteRequestedAt: string }`

6.2 Mock Data

Provide realistic sample data that matches the backend response schemas exactly:

- At least 3 pre-existing proposals in different states (`DRAFT`, `ASSESSED`, `COMPLETED`) matching the wireframe examples ("Benchmarking Util", "My Test Proposal", etc.). Each proposal object must include all fields from the `Proposal` interface (Section 7), with `summary`, `pdfUrl`, `quoteRequestedAt` set to `null` for `DRAFT` proposals
- Hardcoded clarifying questions returned by the assess mock: `["What is the project budget?", "What is the timeline?", "What are the key deliverables?", "What is the target audience?", "Are there compliance requirements?"]` — these must match the backend's hardcoded question set exactly
- A sample PDF file or PDF URL for the Review tab display (used as the `pdfUrl` in `COMPLETED` proposals)
- Sample documents array for pre-existing proposals, with each document containing all fields from the `ProposalDocument` interface

6.3 Mock Service Functions

Implement mock versions of all API calls with signatures and return types matching the API contract table above:

- **`getProposals()`**: Returns `{ proposals: Proposal[] }` after a simulated delay (500ms). The proposals array must be sorted by `updatedAt` descending
- **`createProposal(body: { name: string })`**: Creates a new proposal with a generated UUID, status `DRAFT`, and current ISO timestamps for `createdAt`/`updatedAt`. Returns the full `Proposal` object
- **`getDocuments(proposalId: string)`**: Returns `ProposalDocument[]` for the given proposal
- **`uploadDocuments(proposalId: string, files: File[])`**: Validates file constraints (type: `.pdf`/`.doc`/`.txt`, max 5MB each, max 10 cumulative). On success, returns the full `ProposalDocument[]` for the proposal (including newly uploaded files). On failure, throws an error matching the backend's error response format
- **`assessProposal(proposalId: string, body: { summary: string })`**: Validates summary word count (max 1000 words). Returns `{ questions: string[], stub: string, timerStartedAt: string }` after a simulated delay (1500ms). Sets `timerStartedAt` to the current ISO timestamp
- **`answerQuestions(proposalId: string, body: { answers: string[] })`**: Validates the timer has not expired (15 minutes from `timerStartedAt`). Returns `{ pdfUrl: string }` after a simulated delay (2000ms). Updates proposal status to `COMPLETED`
- **`requestQuote(proposalId: string)`**: Validates proposal status is `COMPLETED`. Returns `{ id: string, status: "QUOTE_REQUESTED", quoteRequestedAt: string }`. Updates proposal status to `QUOTE_REQUESTED`

6.4 Mock Error Responses

Mock services must simulate the same HTTP error codes and messages the backend will return, so error handling UI can be developed and tested:

Scenario	HTTP Status	Error Message
Proposal not found	404	"Proposal not found"
File type not allowed	400	"File type not allowed. Accepted types: .pdf, .doc, .txt"
File exceeds 5MB	400	"File exceeds maximum size of 5MB"
Document count exceeds 10	400	"Maximum of 10 documents per proposal exceeded"
Summary exceeds 1000 words	400	"Summary must not exceed 1000 words"
Timer expired (answer after 15 min)	408	"The assessment context has expired. Please re-assess the proposal."
Proposal not in required status	400	"Proposal must be in {REQUIRED_STATUS} status"

Throw errors with `status` and `message` properties so the UI can differentiate error types and display appropriate user-facing messages.

6.5 Mock Toggle

- Implement mock services behind an environment flag or configuration constant (e.g., `USE_MOCK_API = true`)
- The service layer exports a single set of functions with fixed signatures (as defined in the API Contract table). Internally, these functions delegate to either mock implementations or real API calls based on the flag
- **Critical**: UI components must only call service functions — they must never reference mock data or API URLs directly. This ensures the Integration challenge can swap mock→real implementations with zero component changes

Important Notes

App Isolation

1. **All new code must reside under `src/apps/rfp-tool/`**. This is a self-contained app module that will host both Proposal Management (this challenge) and Admin Challenge Curation (future Challenge 2)
2. **Do NOT modify code in any other app module** (`src/apps/engagements/`, `src/apps/talent-search/`, etc.). The only exception is the route registration in `src/apps/platform/src/platform.routes.tsx`
3. **Do NOT modify shared libraries** (`src/libs/ui/`, `src/libs/core/`, `src/libs/shared/`). If an existing component does not fit, wrap or extend it within the rfp-tool module
4. Reuse existing UI components by importing them — do not copy/duplicate component code into the rfp-tool module

Design Fidelity

1. Follow the wireframe layouts and element positioning from `RFP_Wireframes.pptx` (all 30 elements)
2. Use consistent spacing, typography, and colors from the existing `platform-ui` design system
3. Replicate the exact split-pane layout structure: left sidebar with proposal list, right content area with Build/Review sub-tabs

Technology Constraints

1. Use only React, TypeScript, and libraries already in `platform-ui`

2. Reuse the in-house UI component library (`src/libs/ui`) as mapped in the Reusable Component Mapping table (Section 5)
3. Follow SCSS Modules pattern for styling (`.module.scss` files co-located with components)
4. Follow existing project conventions for file naming, exports, and module structure

Code Quality

1. Well-organized, modular component architecture
2. Clear TypeScript interfaces for all data structures
3. JSDoc comments for exported functions and complex logic
4. No code duplication; extract shared logic into hooks or utilities
5. All components must be properly typed (no `any` types)

Browser Compatibility

- Chrome/Edge (latest 2 versions)
- Firefox (latest 2 versions)
- Safari (latest 2 versions)

Testing

1. Unit tests for all components using React Testing Library
2. Test the mock service functions return expected data
3. Test form validation (file constraints, word limits, timer expiration)
4. Test tab navigation and state transitions
5. Test the countdown timer behavior

AI Review Requirements

Note for Competitors and Reviewers:

All submissions will be evaluated by three automated AI reviewers:

1. **AI Submission Screener** - Validates requirements completeness and best practices
2. **SAST Analysis (Semgrep/OpenGrep)** - Identifies security vulnerabilities and code quality issues
3. **Vulnerability & Secrets Scanner** - Detects dependency vulnerabilities and accidentally committed secrets

Competitors should ensure their submissions pass these AI reviews before final submission. Reviewers must consider the AI review results as part of their evaluation, particularly for security and code quality concerns.

Submission Deliverables

Submit a git patch against the latest commit of the develop branch containing:

1. **Source Code**: All frontend components, services, models, and mock data under `src/apps/rfp-tool/`
2. **Route Registration**: The only change outside `src/apps/rfp-tool/` — updated `platform.routes.tsx` to include the rfp-tool module
3. **Tests**: Jest + React Testing Library tests for all components and mock services
4. **Documentation**: Updated README.md with setup instructions for the proposals feature
5. **Verification evidence**: Screenshots showing all UI states (sidebar with proposal list, Build tab with all three panels, Review tab with incomplete and complete states, timer running and expired), and passing test output